

EECS 841: Computer Vision

Lecture 5

Instructor: Sumaiya Shomaji,
Assistant Professor,

Dept. of EECS, University of Kansas

Email: shomaji@ku.edu

Office: 2020 Eaton Hall and 251 Nichols Hall



Outline (Chapter 3)

Lecture 5_ Point Transform _Part I

Image Transformation: Different ways to transform an image into another image

- Geometric transform
- Point transform

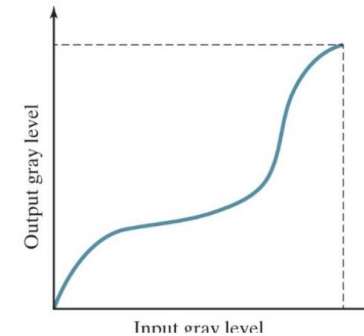
Point Transform in Graylevel

Graylevel Transformations

- **Point transformation:** changes a pixel's value without changing its location.

$$I'(x, y) = f(I(x, y))$$

Figure 3.7 A graylevel transformation maps input gray levels to output gray levels. Based on http://www.unit.eu/cours/videocommunication/Point_Transformation_histogram.pdf



ALGORITHM 3.5 Transform gray levels of an image

TRANSFORMGRAYLEVELS(I, f)

Input: grayscale image I , graylevel mapping f

Output: transformed image I'

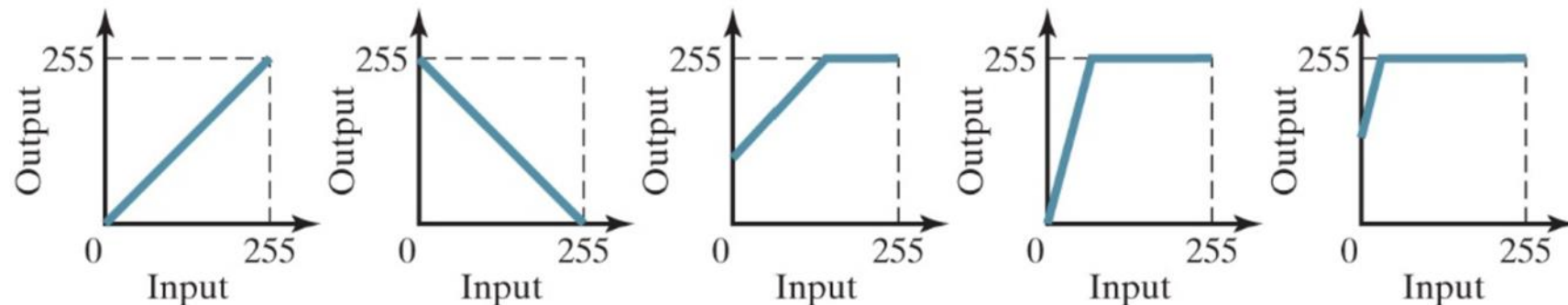
```
1 for  $(x, y) \in I$  do
2    $I'(x, y) \leftarrow f(I(x, y))$ 
3 return  $I'$ 
```

Graylevel Transform: Arithmetic Operation

Arithmetic Operations

- A useful class of gray level transformations is the set of arithmetic operations, depicted in graphical form in the following figure:

Figure 3.8 Arithmetic graylevel transformations. From left to right: identity, inversion, addition (bias), multiplication (gain), and gain-bias transformation, where saturation arithmetic prevents the output from exceeding the valid range. Note that the slope remains 1 under addition, while the mapping passes through the origin under multiplication.



Graylevel Transformation

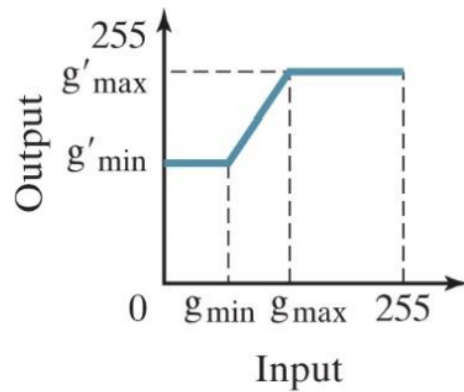
Linear Contrast Stretching

- **Linear contrast stretch:** A transformation that specifies a line segment that maps gray levels between $g_{\min}$ and $g_{\max}$ in the input image to the gray levels $g'_{\min}$ and $g'_{\max}$ in the output image according to a linear function:

$$I'(x, y) = \frac{g'_{\max} - g'_{\min}}{g_{\max} - g_{\min}}(I(x, y) - g_{\min}) + g'_{\min}$$

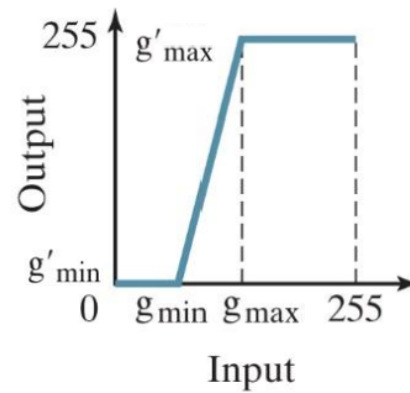
Graylevel Transformation

Linear Contrast Stretching (cont'd)



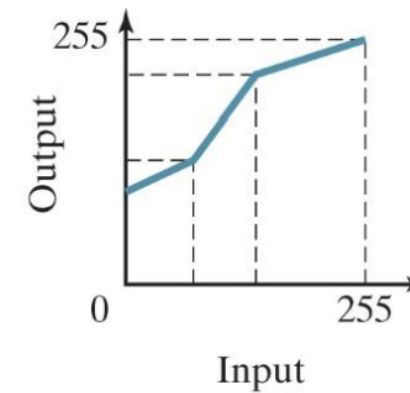
Graylevel Transformation

Linear Contrast Stretching (cont'd)



Graylevel Transformation

Linear Contrast Stretching (cont'd)



$g'_{\min}$ Customized
 $g'_{\max}$ for every
segment

Graylevel Transformation

Linear Contrast Stretching (cont'd)

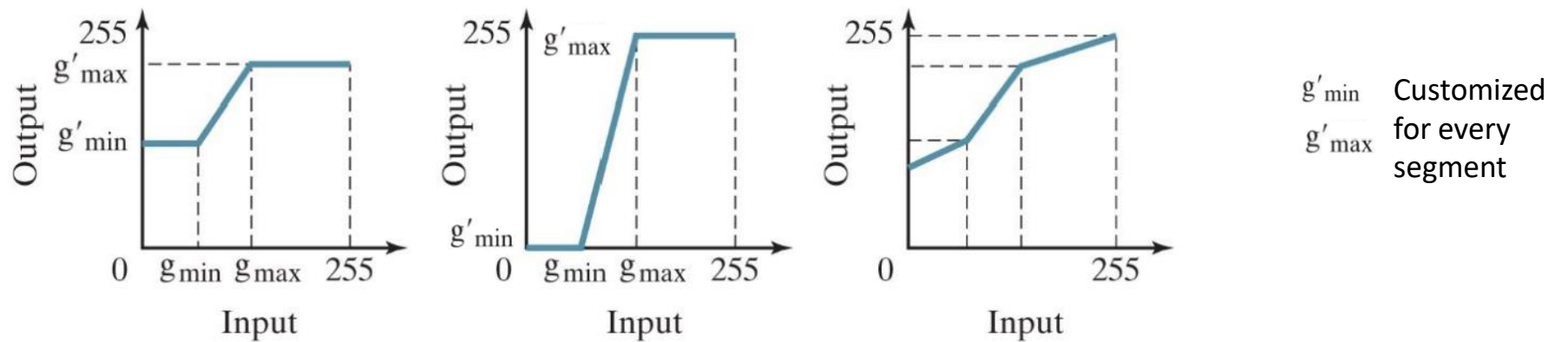


Figure 3.11 Left: Linear contrast stretching maps all the gray levels between $g_{\min}$ and $g_{\max}$ to the range $g'_{\min}$ to $g'_{\max}$. Middle: If $g'_{\min} = 0$ and $g'_{\max} = 255$, then the full output range is used. RIGHT: A piecewise linear contrast stretch can model any graylevel transformation with arbitrary precision.

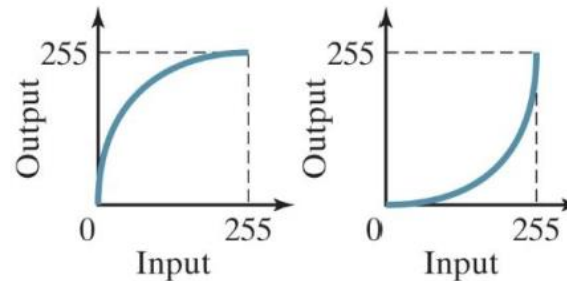
Graylevel Transformation

Analytic Transformations

- Graylevel transformations can be specified using analytic functions such as the logarithm, exponential, or power functions:

$$I'(x, y) = \log(I(x, y))$$

$$I'(x, y) = \exp(I(x, y))$$



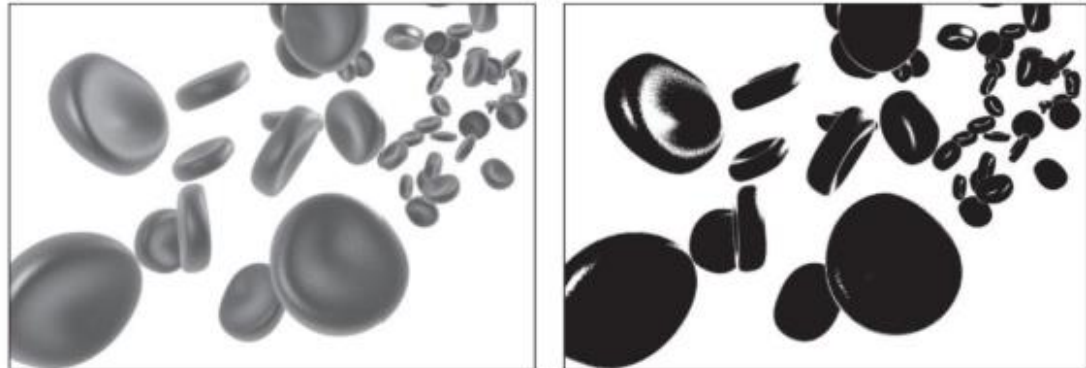
L to R: Logarithmic and
Exponential transformation

Thresholding

- **Thresholding:** takes a grayscale image and sets every output pixel to 1 if its input gray level is above a certain threshold, or to 0 otherwise:

$$I'(x, y) = \begin{cases} 1 & \text{if } I(x, y) > \tau \\ 0 & \text{otherwise} \end{cases}$$

Figure 3.14 An 8-bit grayscale image (left), and the binarized result obtained by thresholding with $\tau = 150$ (right).



Other Transformations

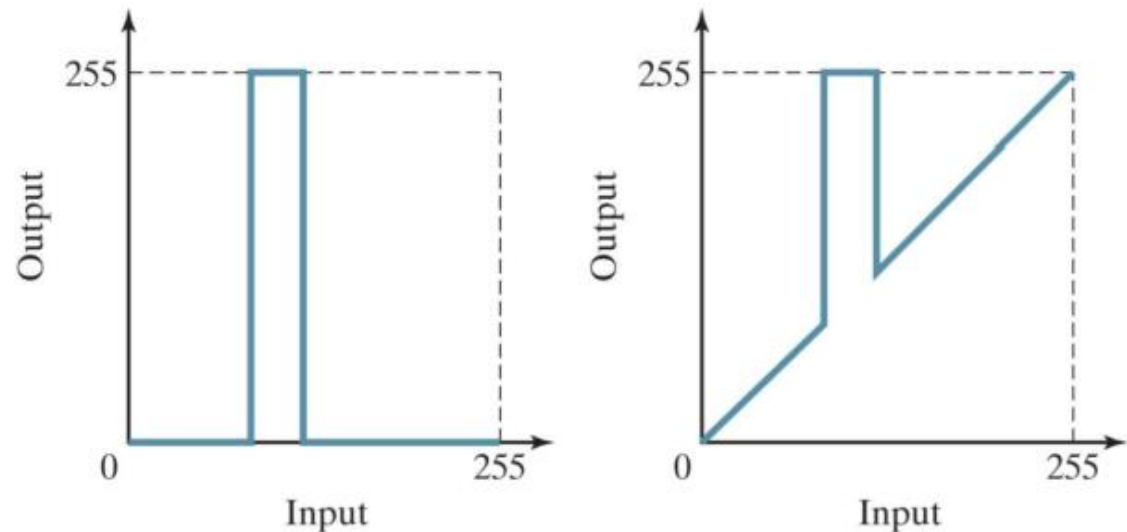
- **Density slicing:** assigns all gray levels within a certain range to a certain value.
- **Quantization:** discards one or more of the lower-order bits.

Other Transformations: Details

Density Slicing

assigns all gray levels within a certain range to a certain value.

Figure 3.15 Density slicing maps a range of input gray levels to a specific output gray level. All other input gray levels are either mapped to zero (left) or remain unchanged (right).



Other Transformations: “n-bit quantization” Details

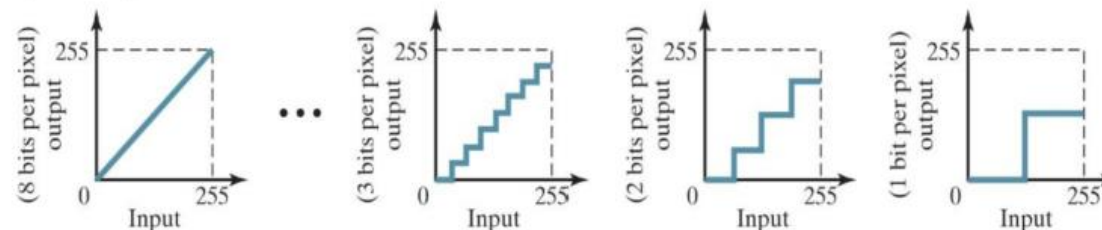
Quantization

- A pixel can have a maximum value of “255”
- How many bits do we need to represent a pixel then?
- Ans:
- 8 bits! Because decimal “255” takes 8 ones to represent that in binary.

Decimal (255) $\rightarrow$ 1111 1111 (Binary)

- During quantization, instead of using 8 bits, we choose fewer number of bits to represent the same pixel (which used to be presented in 8 bits originally)
- We can choose 7 bits, 6 bits, 5 bits, 4 bits, 3 bits, 2 bits, or even 1 bit for quantization!

Figure 3.16 Quantization discards the lower-order bits via a staircase function, with the number of stairs determined by the number of bits retained. From right to left: Only 1 bit is retained, so the gray levels in the dark half (less than 128) map to 0, while the gray levels in the bright half (above 127) map to 128 (binary: 10000000); 2 bits are retained, so gray levels are mapped to either 0, 64, 128, or 192; 3 bits are retained, so all gray levels are mapped to either 0, 32, 64, 96, 128, 160, 192, or 224; all 8 bits are retained (no quantization, the identity function).



Other Transformations: Details

Quantization

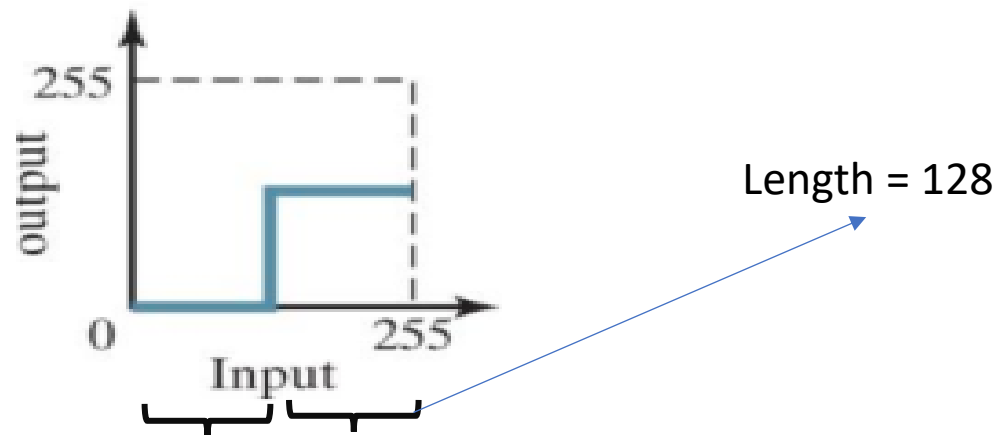
Key-idea: The n in “ n -bit quantization” tells us how many level is permitted in output.

The simplified idea behind n bit quantization is:

- Decide upon what n you want → once n is decided, understand that 2^n levels are permitted in output levels (because we can make only 2^n combinations with n bits).
- Once the permitted output level ($=2^n$) is known, we divide the whole input range in 2^n many groups.
- And the output level intensity for each group is equal to the minimum number from that group in the input level.

What happens when we choose 1 bit during quantization

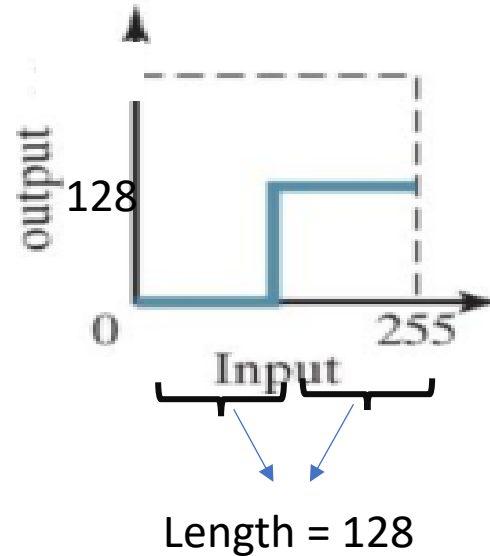
- 1 bit quantization means $n = 1$
- $n = 1$ means we can have $2^1 = 2$ levels.
- Thus, during 1-bit quantization, a pixel can have only 2 possible outcome levels in output image.
- Since there are two possibilities in output, the input range has also been divided by two segments so that if an input pixel lies in one of the segments, we can assign a particular output for that.
- So, $256/2 = 128 = \text{length of each segment} \rightarrow$ therefore the first segment will be $0 \rightarrow 127$ and second will be $128 \rightarrow 255$.
-



What happens when we choose 1 bit during quantization

Finally, the assignment:

- Thus, if an input pixel has an intensity of $0 \rightarrow 127$, we assign “0” for that in output. Similarly, if the input pixel has a value of $128 \rightarrow 255$, we assign “128” for that.



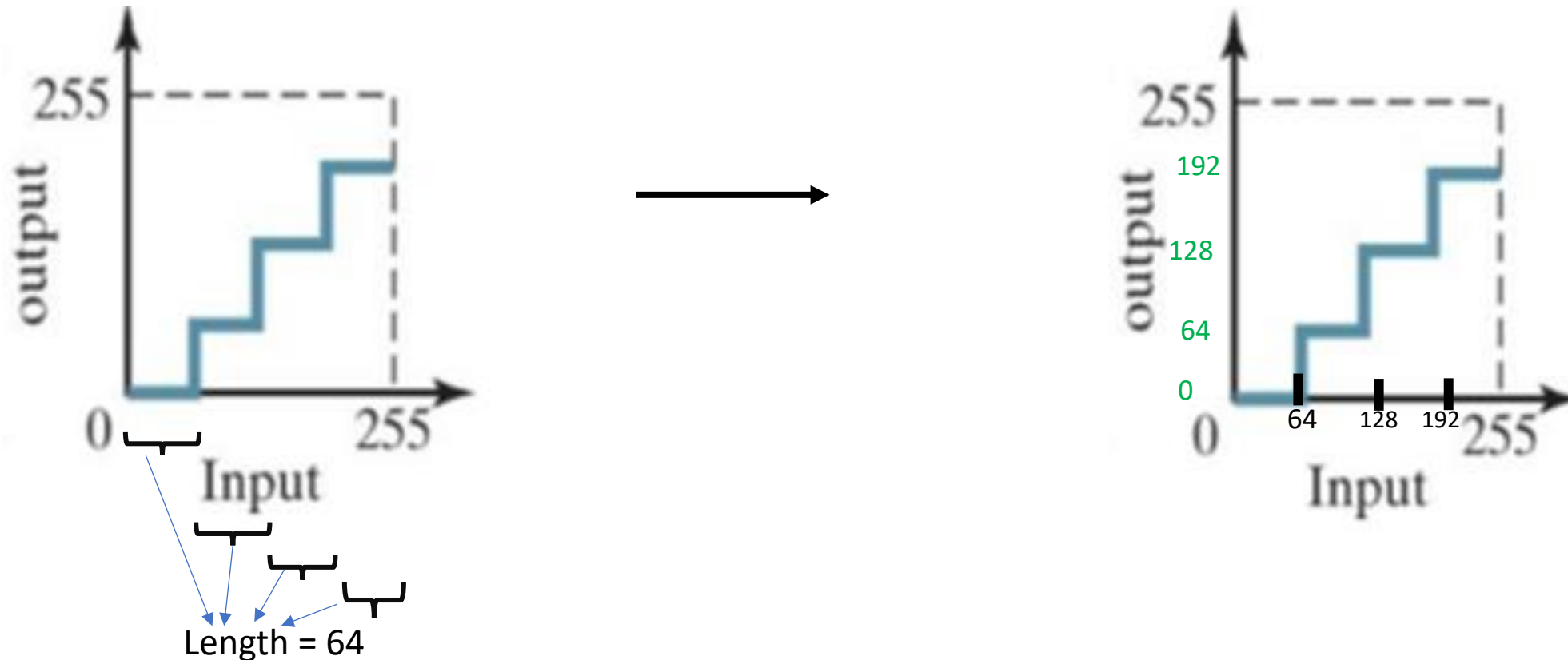
What happens when we choose 2 bit during quantization

Quantization

- 2-bit quantization means $n = 2$
- $n = 2$ means we can have $2^2 = 4$ levels during 2 bit quantization
- Thus, in output, 4 different possibilities can be observed, the pixel will have 4 unique values.
- Since there are 4 possibilities in output, the input range has also been divided by 4 segments so that if an input pixel lies in one of the segments, we can assign a particular output for that.
- So, $256/4 = 64$ = length of each segment $\rightarrow$ therefore the first segment will be $0 \rightarrow 63$, second will be $64 \rightarrow 127$, third will be $128 \rightarrow 191$, and fourth will be $192 \rightarrow 255$.

What happens when we choose 2 bit during quantization

- Thus, if an input pixel has an intensity of $0 \rightarrow 63$, we assign "0" for that in output.
- if an input pixel has an intensity of $64 \rightarrow 127$, we assign "64" for that in output.
- if an input pixel has an intensity of $128 \rightarrow 191$, we assign "128" for that in output.
- if an input pixel has an intensity of $192 \rightarrow 255$, we assign "192" for that in output.



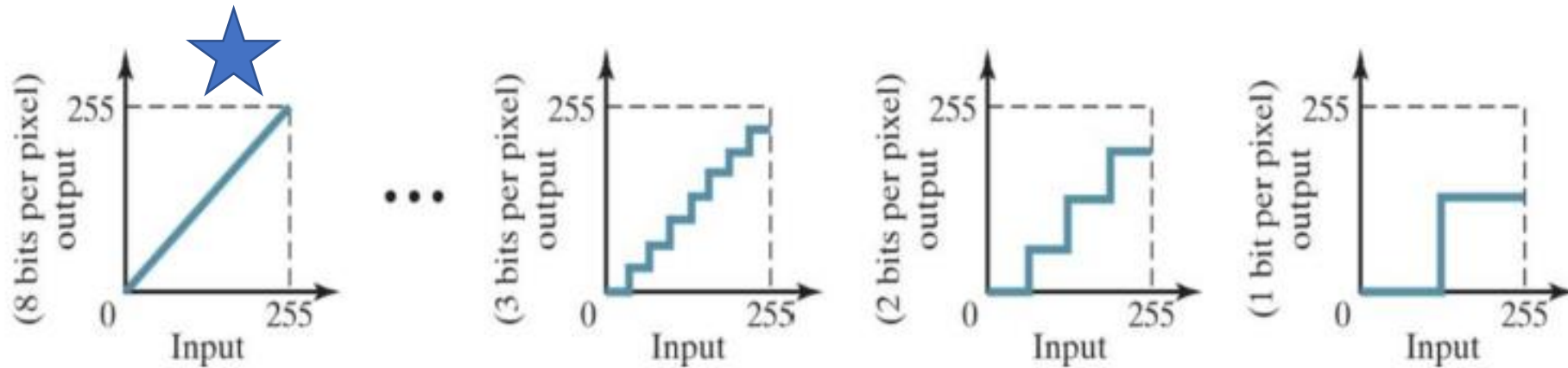
Other Transformations: 3-bit quantization

Quantization

Question: If an input pixel has intensity value of “158” , what should be the corresponding output intensity value for 3-bit quantization?

Other Transformations: 8 bit quantization

- Not a “quantization” though!



Other Transformations: Details

Figure 3.17 From left-to-right and top-to-bottom: An original 8-bit-per-pixel image of a fire hydrant and its quantized versions with 7, 6, 5, 4, 3, 2, and 1 bit per pixel. Note that the image is quite recognizable with as few as 3 bits per pixel.

